for-else while-else in Python

Outcomes

- Can we use **else** with Looping statements?
- When else will be executed in **for-else?**
- When else will be executed in **while-else?**
- Applications and implementation

else with for/while?

- Can we write

else with for

else with while

Yes

for-else and while-else

- In Python, the else block isn't just for if statements.
- You can also use it with for and while loops.
- This is a **unique feature** that often confuses beginners, but it is incredibly useful once you understand its "Rule."

The Rule of Loop-Else

- The else block executes **ONLY IF** the loop completes its entire cycle naturally (without being interrupted by a break statement).
- Think of the else block as a **"Success Message"** that says: *"I finished the whole job without stopping early!"*

for-else

- The else clause in a for loop will be executed when the loop **completes all the iterations normally.**

  i.e., without encountering a break statement.

Applications

- The for-else structure is most commonly used for **searching**.
- If you look through a list and find what you need, you break.
- If you look through the *entire* list and don't find it, the else block triggers.

Real-Life Example: Searching for a Key

- You have a bag of 5 keys.
- You try each one to open a door.
- **If a key works:** You open the door and stop trying (break).
- **If you try EVERY key and none work:** You realize you are at the wrong house (else).

else with while

- else clause in a while loop will be executed when the loop completes normally

  i.e., **Without encountering a break statement.**

Real-Life Example: Phone Battery

- Your phone stays on as long as the battery is above 0%.
- **Normal flow:** Battery hits 0% → Phone shuts down gracefully (else).
- **Interrupted flow:** You manually turn the phone off (break) → The "Low Battery" shutdown process is skipped.

Summary Comparison

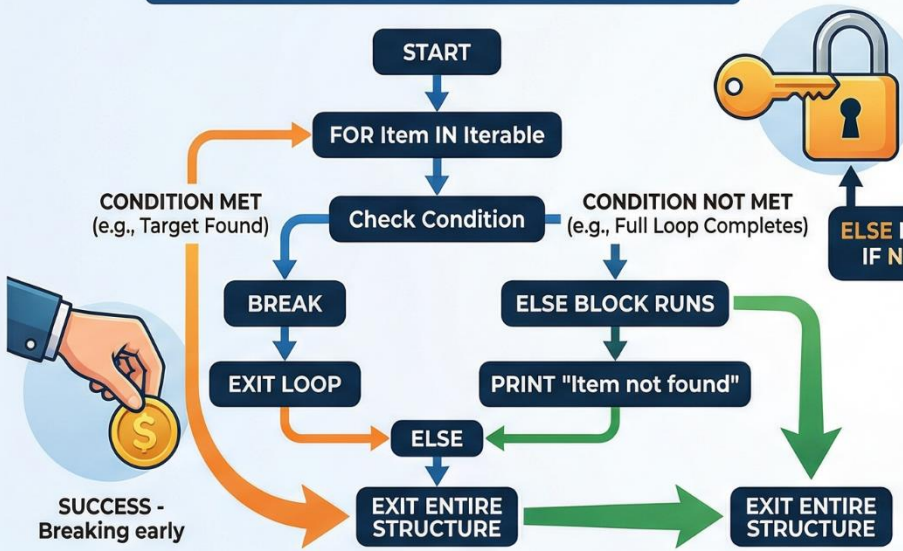
Scenario	Did the loop hit break?	Does else execute?
Search Successful (element found)	Yes	**No**
Search Failed (Finished List)	No	**Yes**
While Condition becomes False	No	**Yes**
Loop forced to stop early	Yes	**No**

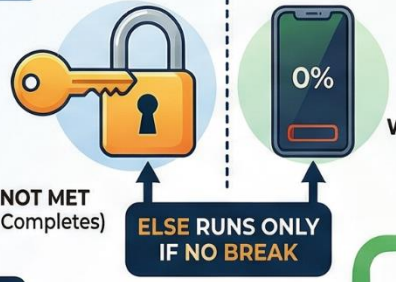
Pro-Tip

- If you find for-else confusing, you can think of it as a for-if_not_found block. It helps you avoid using "flag" variables (like found = False) which makes your code much cleaner!
- Does this "Success Message" analogy help you visualize when the else block kicks in?

DEMYSTIFYING THE 'ELSE' IN PYTHON LOOPS

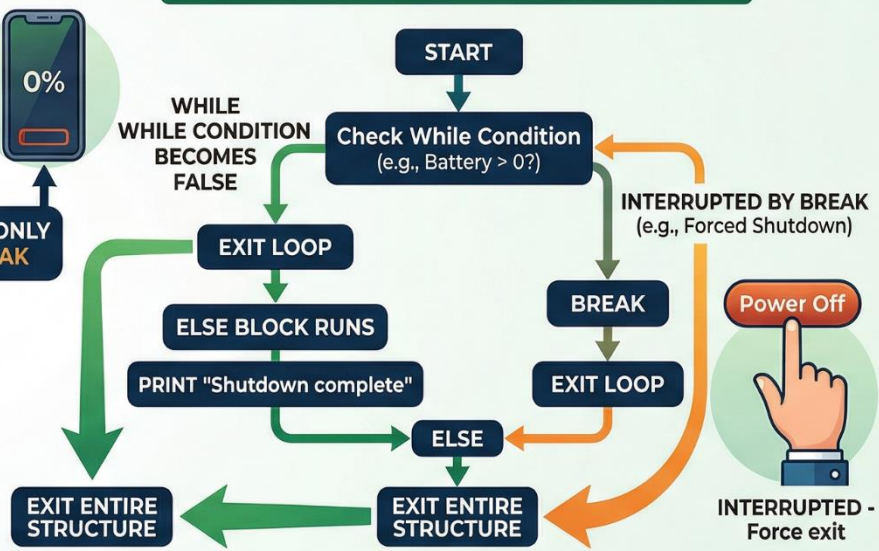
FOR-ELSE
(SEARCHING SCENARIO)






```
items = ["apple", "banana", "cherry"]
search = "kiwi"  # not found
for item in items:
    if item == search:
        print("Found!")
        break
else:
    print("Not Found!") # Runs
```

WHILE-ELSE
(CONDITION DECAY SCENARIO)




```
battery = 5
while battery > 0:
    print(f"ON: {battery}%")
    battery -= 1
    if force_quit:
        break
else:
    print("Battery empty. Normal shutdown.") # Runs
```

Check your Knowledge

Can we use **else** with Looping statements?

When else will be executed in for-else?

When else will be executed in while-else?

Applications and implementation